Test 1 Practice

1) Make **corrections** to the following code to fix any syntax or logical errors that would prevent it from either executing or providing the intended answer. Points will be deducted for incorrect changes that "break" the code.

```
See key on Moodle
```

2) How many 'B's are output?

```
b = "BBB"
for x in range(2, 4):
   for y in range(9, 7, -1):
      b = b + "B"
      print("B")
   print("B")
print(b)
```

```
13
```

3) What is the output of the following?

a)

```
x = ["Willie", "Jonas", "Roy", "Eunice"]
gilligan = x[1: ][2].find("e")
print(gilligan)
```

```
5
```

b)

```
a = 2
b = 7
while b > 3:
   for a in range(1, 5, 1):
      b = b - 2
      if a >= b:
         a = a * 2
         print(f"{a:5}", end = " ")
   print(f"{a*1000:10,}")
print(f"{b=}")
```

```
    6     8      8,000
b=-1
```

4) Write a function called `add_up` that is passed no parameters. The function adds positive integers entered by the user until a -1 is entered. The function returns the sum of all numbers added aside from the final -1.

```
def add_up():
    x = 1
    total = 0
    x = int(input("Enter a positive integer or -1 to stop: "))
    while x != -1:
        total += x
        x = int(input("Enter a positive integer or -1 to stop: "))
    return total
```

5) Write a function called `char_dict` that is passed a string and returns a dictionary that contains key-value pairs of individual characters in the string and the number of times which they occur. For example, the function call `char_dict("Snow, snow, go away!")` would return:

```
{'S': 1, 'n': 2, 'o': 3, 'w': 3, ',': 2, ' ': 3, 's': 1, 'g': 1, 'a': 2, 'y': 1, '!': 1}
```

```
def letter_dict(text):
    d = {}
    for ch in text:
        if ch in d:
            d[ch] += 1
        else:
            d[ch] = 1
    return d
```

6) You may recognize the following password validation function from Lecture 06.

```
def valid_password(password):
    correct_length = False
    has_uppercase = False
    has_lowercase = False
    has_digit = False

    if len(password) >= 7:
        correct_length = True

        for ch in password:
            if ch.isupper():
                has_uppercase = True
            if ch.islower():
                has_lowercase = True
            if ch.isdigit():
                has_digit = True

    return correct_length and has_uppercase and has_lowercase and has_digit
```

Write code that made the following modifications to the password:

(a) If the function returns False, concatenate `"abcABC123"` to the end of the password.

```
if not valid_password(password):
    password = password + "abcABC123"
```

(b) If the function returns False, for each uppercase character in the password, insert the corresponding lowercase character after it, and vise-a-versa. Concatenate the original length of the password to the end. For example: `"PASSwd"` would become `"PpAaSsSswWdD6"`. If the original length is less than 3 or if it contains no alphabetical characters, add `"Ab12"` to the front of it.

```
if not valid(password):
     has_alpha = False
     olen = len(password)
     newpw = ""

     for ch in password:
          if ch.isupper():
               newpw += ch + ch.lower()
               has_alpha = True
          elif ch.islower():
               newpw += ch + ch.upper()
               has_alpha = True
          else:
               newpw += ch
     newpw += str(olen)
     if olen < 3 or not has_alpha:
          newpw = 'Ab12' + newpw
     password = newpw
print(newpw)
```

(c) If the function returns True, reverse the order of the characters in the password.

```
     password = password[::-1]
```